Projet : clustering sous contrainte d'ordre

à rendre au plus tard le 27/11/2026 (FI) et le 04/12/2026 (FA)

1 Objectif

Ce projet vise à implémenter une méthode de clustering permettant de partitionner une séquence de données en classes homogènes et ordonnées dans le temps : c'est la méthode de programmation dynamique de R. A. Fisher. Celle-ci est utilisée pour découper automatiquement des signaux temporels, ou pour détecter des ruptures dans ces séquences. Dans le domaine de la reconnaissance de la parole par exemple, cette méthode peut être exploitée pour partitionner des signaux audio en intervalles de temps associés à des locuteurs différents. De manière plus générale, la segmentation temporelle est un outil pertinent pour structurer des données présentant un ordre temporel ou spatial. La figure 1 illustre sur des données 1D la différence entre un clustering classique et un clustering sous contrainte d'ordre.

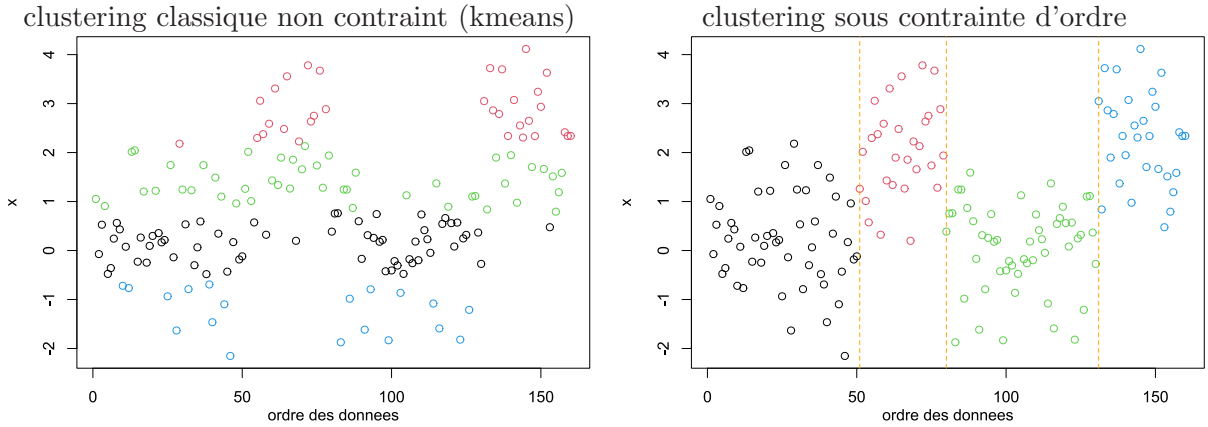


FIGURE 1 – Partitionnement de données avec contraintes d'ordre sur les classes (les traits en pointillés correspondent aux limites des vraies classes)

2 Algorithme de programmation dynamique de Fisher

L'algorithme de programmation dynamique de Fisher cherche à partitionner une séquence de données $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n)$, avec $\boldsymbol{x}_i \in \mathbb{R}^p$, en K **classes se succédant dans le temps**. Compte-tenu de la contrainte d'ordre imposée aux classes, ce problème diffère de celui (classique) visant à trouver une partition minimisant par exemple l'inertie intra-classe. L'objectif est désormais de déterminer K intervalles temporels $[1; t_1 - 1]$, $[t_1; t_2 - 1]$, $\cdots$, $[t_{K-2}; t_{K-1} - 1]$, $[t_{K-1}; n]$. De manière équivalente, ce problème consiste à trouver $K-1$ instants de changement $(t_k)_{k=1,\ldots,K-1}$ par minimisation du critère (de type inertie intra-classe) suivant :

$$\begin{cases} C(t_1, \ldots, t_{K-1}) &= D(1, t_1 - 1) + \left(\sum_{k=2}^{K-1} D(t_{k-1}, t_k - 1)\right) + D(t_{K-1}, n) & \text{si } K \geq 3 \\ C(t_1) &= D(1, t_1 - 1) + D(t_1, n) & \text{si } K = 2, \end{cases} \tag{1}$$

avec

$$D(a, b) = \sum_{i=a}^{b} \sum_{j=1}^{p} (\boldsymbol{x}_i^j - \boldsymbol{\mu}_{ab}^j)^2 \qquad \text{où} \quad \boldsymbol{\mu}_{ab}^j = \frac{\sum_{i=a}^{b} \boldsymbol{x}_i^j}{b - a + 1}. \tag{2}$$

On notera ici que $\boldsymbol{\mu}_{ab}^{j}$ représente la moyenne de la sous-séquence $(\boldsymbol{x}_a, \ldots, \boldsymbol{x}_b)$ suivant la $j$ème variable. On remarquera aussi que D est une matrice (triangulaire supérieure) de taille (n, n) puisque $D(a, b)$ n'est défini que pour $b \geq a$.

Contrairement au critère d'inertie intra-classe qui a été étudié en cours (dans lequel aucune contrainte d'ordre n'est imposée sur les classes), le critère C peut ici être **minimisé de manière exacte (optimum global)** grâce à l'algorithme de programmation dynamique de Fisher qui est basé sur le principe selon lequel « toute sous-partition d'une partition optimale est nécessairement optimale ». L'algorithme est décrit en page 3.

3 Travail à effectuer

1) Implémenter sous R ou Python l'algorithme de programmation dynamique de Fisher (voir page 3). Créer pour cela deux fonctions : une fonction `diam` calculant le diamètre $D(a, b)$ suivant la formule (2) et une fonction `clustfisher` pour l'algorithme décrit en page 3. Pour le calcul de l'argument minimum (argmin) d'un vecteur numérique, vous pourrez utiliser la fonction `which.min` de `R`; et sous python, vous pourrez utiliser `np.argmin` si vous travaillez avec `numpy`.

2) Application à des données simulées bi-dimensionnelles : en utilisant l'algorithme de Fisher, segmenter la séquence de données simulées `sequencesimu` (à télécharger) et régler le nombre de classes (segments) à l'aide de la méthode du coude appliquée au critère C. Comparer cette partition à celles issues de deux autres algorithmes de votre choix étudiés en cours.

3) Application à des données réelles : segmenter en 4 classes la séquence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{140})$ des données `aiguillage` (on rappelle dans ce cas, que chaque observation $\boldsymbol{x}_i$ appartient à $\mathbb{R}^{552}$. Comparer la partition obtenue par l'algorithme de Fisher avec la vraie partition (553ème colonne du tableau) ainsi qu'avec les partitions issues de deux autres algorithmes de votre choix. Commentez les résultats obtenus.

4 Documents à rendre

Un compte-rendu n'excédant pas 10 pages (.doc ou .pdf) et le code R ou Python associé.

Algorithme 1 : Algorithme de programmation dynamique de Fisher

Entrées : séquence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n)$, nombre de segments souhaité ($K \geq 1$)

Étape 0

Initialiser les matrices et vecteurs suivants (*utiliser des matrices et des vecteurs nuls*) :

D	(dimension $n \times n$)
M	(dimension $n \times K$)
A	(dimension $n \times K$)
t	(longueur $K-1$)
$cluster$	(longueur n)

Étape 1 : calcul de la matrice triangulaire supérieure D

pour $a = 1, \ldots, n,\ b = a, \ldots, n$ **faire**
| Calculer $D[a, b]$ selon la formule (2)
fin

Étape 2 : calcul récursif des critères optimaux

pour $i = 1, \ldots, n$ **faire**
| $M[i, 1] \leftarrow D[1, i]$
fin
pour $k = 2, \ldots, K$, $i = k, \ldots, n$ **faire**
| $M[i, k] \leftarrow \min\Big\{M[t-1, k-1] + D[t, i] \quad \text{avec} \quad t = k, \ldots, i\Big\}$
| $A[i, k] \leftarrow \operatorname{argmin}\Big\{M[t-1, k-1] + D[t, i] \quad \text{avec} \quad t = k, \ldots, i\Big\}$
fin

Étape 3 : calcul récursif des points de rupture optimaux

$k \leftarrow K - 1$
$m \leftarrow n$
tant que $k \geq 1$ **faire**
| $t[k] \leftarrow A[m, k+1]$
| $m \leftarrow t[k] - 1$
| $k \leftarrow k - 1$
fin

Étape 5 : critère d'inertie C et labels des classes

$C \leftarrow M[n, K]$
pour $i = 1, \ldots, t[1] - 1$ **faire**
| $cluster[i] \leftarrow 1$
fin
pour $k = 2, \ldots, K-1$, $i = t[k-1], \ldots, t[k] - 1$ **faire**
| $cluster[i] \leftarrow k$
fin
pour $i = t[K-1], \ldots, n$ **faire**
| $cluster[i] \leftarrow K$
fin

Sorties : points de rupture $t = (t_1, \ldots, t_{K-1})$, labels $cluster$, critère d'inertie C